```
#include <iostream>
#include <algorithm>
#include <iomanip>
using namespace std;
struct Item {
    int id, weight, value;
    double ratio;
};
bool compare(Item a, Item b) {
    return a.ratio > b.ratio;
}
int main() {
    int n, capacity;
    cout << "Enter number of items: ";
    cin >> n;
    Item items[50];
    cout << "Enter weight and value of each item:\n";
    for (int i = 0; i < n; i++) {
        items[i].id = i + 1;
        cin >> items[i].weight >> items[i].value;
        items[i].ratio = (double)items[i].value / items[i].weight;
    }
    cout << "Enter knapsack capacity: ";
    cin >> capacity;
    stable_sort(items, items + n, compare);
    cout << fixed << setprecision(2);
    cout << "\nItems sorted by profit/weight ratio (highest first):\n";
    for (int i = 0; i < n; i++)
        cout << "Item " << items[i].id << ": weight = " << items[i].weight << ", profit = "
                << items[i].value << ", ratio = " << items[i].ratio << endl;
    double totalValue = 0;
    for (int i = 0; i < n && capacity > 0; i++) {
        cout << "\nStep " << i + 1 << ": Item " << items[i].id << " (weight = " << items[i].weight
                << ", profit = " << items[i].value << ")\n";
        double gain;
        if (items[i].weight <= capacity) {
            capacity -= items[i].weight;
            gain = items[i].value;
            cout << "Whole item taken: " << items[i].weight << " of " << items[i].weight << " units\n";
        } else {
            gain = items[i].ratio * capacity;
            cout << "Part of item taken: " << capacity << " of " << items[i].weight
                    << " units (fraction = " << (double)capacity / items[i].weight << ")\n";
            capacity = 0;
        }
        totalValue += gain;
        cout << "Profit added = " << gain << endl;
        cout << "Total profit = " << totalValue << endl;
        cout << "Remaining capacity = " << capacity << endl;
        if (i == n - 1) {
            cout << "Remaining items: none\n";
        } else {
            cout << "Remaining items (highest ratio first):\n";
            for (int j = i + 1; j < n; j++)
                cout << "  Item " << items[j].id << ": weight = " << items[j].weight << ", profit = "
                        << items[j].value << ", ratio = " << items[j].ratio << endl;
        }
    }
    cout << "\nMaximum value = " << totalValue << endl;
    return 0;
}
```

```
PS C:\Users\fishb\Desktop\Sem 7\DAA> cd "c:\Users\fishb\Desktop\Sem 7\DAA\" ; if ($?) { g++ 3_fractional_knapsack.cpp -o
3_fractional_knapsack } ; if ($?) { .\3_fractional_knapsack }
Enter number of items: 3
Enter weight and value of each item:
10 60
20 100
30 120
Enter knapsack capacity: 50

Items sorted by profit/weight ratio (highest first):
Item 1: weight = 10, profit = 60, ratio = 6.00
Item 2: weight = 20, profit = 100, ratio = 5.00
Item 3: weight = 30, profit = 120, ratio = 4.00

Step 1: Item 1 (weight = 10, profit = 60)
Whole item taken: 10 of 10 units
Profit added = 60.00
Total profit = 60.00
Remaining capacity = 40
Remaining items (highest ratio first):
  Item 2: weight = 20, profit = 100, ratio = 5.00
  Item 3: weight = 30, profit = 120, ratio = 4.00

Step 2: Item 2 (weight = 20, profit = 100)
Whole item taken: 20 of 20 units
Profit added = 100.00
Total profit = 160.00
Remaining capacity = 20
Remaining items (highest ratio first):
  Item 3: weight = 30, profit = 120, ratio = 4.00
```

```
PS C:\Users\fishb\Desktop\Sem 7\DAA> cd "c:\Users\fishb\Desktop\Sem 7\DAA\" ; if ($?) { g++ 3_fractional_knapsack.cpp -o
3_fractional_knapsack } ; if ($?) { .\3_fractional_knapsack }
Item 1: weight = 10, profit = 60, ratio = 6.00
Item 2: weight = 20, profit = 100, ratio = 5.00
Item 3: weight = 30, profit = 120, ratio = 4.00

Step 1: Item 1 (weight = 10, profit = 60)
Whole item taken: 10 of 10 units
Profit added = 60.00
Total profit = 60.00
Remaining capacity = 40
Remaining items (highest ratio first):
  Item 2: weight = 20, profit = 100, ratio = 5.00
  Item 3: weight = 30, profit = 120, ratio = 4.00

Step 2: Item 2 (weight = 20, profit = 100)
Whole item taken: 20 of 20 units
Profit added = 100.00
Total profit = 160.00
Remaining capacity = 20
Remaining items (highest ratio first):
  Item 3: weight = 30, profit = 120, ratio = 4.00

Step 3: Item 3 (weight = 30, profit = 120)
Part of item taken: 20 of 30 units (fraction = 0.67)
Profit added = 80.00
Total profit = 240.00
Remaining capacity = 0
Remaining items: none

Maximum value = 240.00
```